

SOFTWARE TESTING

Unit III

AI Techniques for Software Testing

Detailed Study Notes · 6 Hours · With Worked Examples

Topics Covered

1. Why AI in Software Testing
2. AI-Driven Test Case Generation
3. ML-Based Defect Prediction
4. AI in Requirement Analysis (NLP)

Contents

- 1. Why AI in Software Testing
 - 1.1 Limitations of Traditional Testing
 - 1.2 Five Key Reasons to Use AI in Testing
 - 1.3 Worked Example — Self-Healing Test Script
 - 1.4 Popular AI-Powered Testing Tools
- 2. AI-Driven Test Case Generation
 - 2.1 What AI Reads (Inputs)
 - 2.2 What AI Produces (Outputs)
 - 2.3 Worked Example — Login Feature
- 3. ML-Based Defect Prediction
 - 3.1 Factors ML Analyses
 - 3.2 The Defect Prediction Pipeline
 - 3.3 Worked Example — Module Risk Scoring
- 4. AI in Requirement Analysis (NLP)
 - 4.1 Core Capabilities
 - 4.2 Worked Example — Requirement to Test Case
- 5. Unit Summary & Quick Revision Notes

1. Why AI in Software Testing

Software applications today are larger, more frequently updated, and tested across more devices and browsers than ever before. Purely manual and traditionally-scripted automated testing struggles to keep pace with this scale of change. Artificial Intelligence (AI) and Machine Learning (ML) are increasingly used to make testing faster, more adaptive, and less dependent on constant manual upkeep.

1.1 Limitations of Traditional Testing

Before looking at what AI brings to testing, it helps to understand the pain points of conventional automated testing that AI is designed to solve:

- **Manual test creation and maintenance:** Test cases are written by hand and must be individually updated whenever the application changes.
- **Fragile locators:** Automation scripts typically refer to UI elements by a fixed ID, name, or XPath. Even a small UI redesign can break dozens of scripts at once.
- **Time-consuming regression cycles:** Large regression suites (sometimes thousands of test cases) take hours to execute and review, slowing down releases.
- **Flaky tests:** Some tests fail intermittently for reasons unrelated to real bugs (timing issues, network delays), and teams waste time investigating false alarms.
- **Coverage limited by human capacity:** The number and variety of test cases a team can design by hand is limited by available time and tester experience.

AI-based testing tools directly target each of these limitations, using machine learning models and natural language processing to make test creation, execution and maintenance considerably more intelligent and self-sufficient.

1.2 Five Key Reasons to Use AI in Testing

i. Handles Large Test Case Volume

Modern applications can require thousands of regression test cases. Running every test on every build is often impractical within release timelines. AI models can analyze code changes, historical defect patterns and test outcomes to intelligently select and prioritize the subset of tests most likely to catch a defect, instead of running the full suite blindly every time.

Example: A banking application maintains a regression suite of 10,000 test cases. Instead of executing all of them on every build (which could take many hours), an AI engine analyses which modules changed and which tests historically catch defects in those modules, then prioritizes and runs the top 500 highest-risk tests first — giving fast feedback without sacrificing coverage of what matters most.

ii. Adapts to Frequent UI Changes

User interfaces change often — buttons are redesigned, elements are repositioned, and IDs or class names are regenerated by frameworks. Traditional scripts that rely on a single fixed locator break the moment any of this changes. AI-based tools instead learn multiple attributes of an element (its text, approximate position, style, and surrounding context) so they can still recognize it even if one attribute changes.

Example: A "Submit" button's HTML ID changes from #submitBtn to #btn_submit_2024 after a front-end redesign. A traditional script that hard-codes the old ID immediately fails with an "Element not found" error. An AI-based tool, however, recognizes the same button by its visible label, its position on the page, and its visual styling, and continues clicking it correctly without any script changes.

iii. Reduces Automation Maintenance

A large fraction of a test automation team's time is typically spent not writing new tests, but repairing existing ones after every UI or workflow change. Because AI-based locators adapt automatically (this is often called "self-healing"), the number of broken scripts after a release drops sharply, freeing testers to focus on designing new test scenarios rather than patching old ones.

Example: After a UI revamp that would traditionally require a team to manually inspect and rewrite around 200 Selenium scripts, an AI-based self-healing framework automatically updates the affected locators itself and simply logs a maintenance report — cutting days of rework down to a quick review of the log.

iv. Reduces Flaky Scripts

A 'flaky' test is one that sometimes passes and sometimes fails without any change to the underlying application — usually because of environment issues like network latency, animation timing, or test-data conflicts. AI models can learn the difference between a failure pattern caused by a genuine defect and one caused by an unstable environment, and can suppress or flag the latter instead of reporting it as a bug.

Example: An AI-driven test platform observes that a particular checkout test fails only when the test environment's network response time exceeds two seconds, and never fails otherwise. It learns this is an environmental issue, not a real defect, and stops flagging it as a failing test — saving the team from investigating a false alarm on every run.

v. Improves Speed, Accuracy and Coverage

Because AI systems can execute and analyze large numbers of test combinations in parallel, and can compare visual output pixel-by-pixel far more consistently than a human reviewer, they improve both the thoroughness and the reliability of testing — surfacing subtle visual or functional defects that might otherwise be missed under time pressure.

Example: A visual-AI tool compares thousands of screenshots of a website across different browsers and screen sizes within minutes, flagging a 2-pixel misalignment in a checkout button on Safari that a human tester manually reviewing screens might easily miss.

1.3 Worked Example — Self-Healing Test Script

The table below traces through what happens to a login button's locator, comparing traditional automation with AI-based automation, when its HTML ID changes from #loginBtn to #login_button_v2 after a UI update:

Step	Traditional Automation	AI-Based Automation
Locator used	id="loginBtn" (hard-coded)	Learns multiple attributes: visible text, position, style
After UI change	Script fails: "Element not found"	AI recognizes the button via similarity matching; auto-updates the locator

Tester action needed	Manually inspect and fix the script	None — the test continues running unattended
Result	Build is blocked until the script is fixed	Test passes; a maintenance log records the auto-update

This self-healing capability is the single biggest reason AI-based tools cut automation maintenance effort so sharply — the tool adapts to change instead of merely reporting failure.

1.4 Popular AI-Powered Testing Tools

A number of commercial and open-source tools now embed AI/ML techniques into their testing workflow. A few widely used examples:

- **Testim:** Uses AI to create stable, self-healing locators for web test automation, reducing script breakage after UI changes.
- **Applitools:** A Visual AI platform that performs pixel-level UI comparisons to detect visual regressions that functional tests would miss.
- **mabl:** A low-code AI testing platform with built-in auto-healing, and analytics that highlight likely causes of failure.
- **Functionize:** Uses ML to author, execute and self-heal tests at scale across large web applications.
- **Test.ai (part of Tricentis):** Uses computer vision and ML to test mobile apps the way a human user would interact with them.
- **Eggplant AI:** Uses AI-driven autonomous exploration of an application to discover defects without needing pre-written scripts.

Summary — Topic 1

AI does not replace testers; it automates the repetitive, high-volume, and maintenance-heavy parts of the testing process. Its biggest structural innovation is self-healing automation, which allows scripts to adapt to UI change instead of breaking. The net result for a QA team is faster releases, fewer false alarms from flaky tests, and broader coverage achieved with the same headcount.

2. AI-Driven Test Case Generation

AI-driven test case generation refers to the use of AI and ML models to automatically read project artifacts — requirements, user stories, defect history and UX flows — and generate relevant test cases, test data and edge cases, without a human manually authoring every scenario from scratch.

Example: Instead of a QA engineer spending a day manually writing 40 test cases for a checkout page, an AI model reads the associated user story and the page's navigation flow, and drafts a comparable (or larger) set of test cases — including some edge cases a human might overlook — within minutes.

Process Overview

The process can be thought of as a simple pipeline: a set of textual and historical inputs feed into an AI engine (typically combining Natural Language Processing with pattern-learning from past test data), which in turn produces structured testing artifacts as outputs.

Step 1 — Inputs are collected: Requirements, user stories, past defect records, and UX/navigation flow diagrams are gathered as the raw material the AI will read.

Step 2 — The AI engine analyses the inputs: NLP techniques extract actions, actors, and conditions from text; pattern-learning models study relationships between past changes and past defects.

Step 3 — Outputs are generated: The engine produces test cases, associated test data, and edge cases that a human reviewer can inspect, adjust and approve.

2.1 What AI Reads — Input Sources

- **Requirements:** Formal statements of expected system behaviour, e.g. "The system shall allow users to reset their password via email." AI parses this to extract the actionable rule that needs to be tested.
- **User Stories:** Short descriptions of a feature from a user's perspective, e.g. "As a customer, I want to filter products by price so I can find items in my budget." AI derives concrete filter test scenarios from the intent expressed here.
- **Past Defects:** Historical bug records. If a previous defect involved a negative price crashing the price filter, AI adds a regression test for that exact scenario so the same bug can never slip through unnoticed again.
- **UX Flow:** Screen-flow or navigation diagrams (e.g. Login → Dashboard → Cart → Checkout). AI maps out the paths a user can take through the application, ensuring test cases cover realistic navigation sequences, not just isolated screens.

2.2 What AI Produces — Output Types

- **Test Cases:** Step-by-step scenarios covering functional, UI and workflow behaviour — for example, "Verify login with valid credentials." These are typically generated with clear preconditions, steps and expected results.

- **Test Data:** Realistic and varied input values generated automatically to exercise the test cases — valid emails, invalid formats, boundary-length passwords, and so on — without a tester having to hand-craft every value.
- **Edge Cases:** Rare but important scenarios that are easy for a human to overlook under time pressure — such as a login attempt with an unusually long password, or a form submitted with every field left empty at once.

2.3 Worked Example — Login Feature

User Story: "As a user, I want to log in with my email and password so that I can access my account."

Given only this one-line user story, an AI test-generation engine can produce a structured set of positive, negative and edge-case tests, complete with sample test data:

Type	Test Case	Sample Test Data
Positive	Login with valid email and correct password	user@test.com / Pass@123
Negative	Login with valid email and incorrect password	user@test.com / wrongPass
Negative	Login with an unregistered email address	unknown@test.com / Pass@123
Edge Case	Login with empty email and password fields	"" / ""
Edge Case	Login with a maximum-length password (255 characters)	user@test.com / "A..." (255 chars)
Edge Case	Login attempt using SQL-injection style input	' OR '1'='1 / anything

Notice that three of the six generated cases are edge cases that a rushed manual test-design pass might easily skip — this is exactly where AI-driven generation adds the most value.

Summary — Topic 2

AI-driven test case generation reads four kinds of inputs (requirements, user stories, past defects, UX flow) and produces three kinds of outputs (test cases, test data, edge cases). Its single biggest benefit is surfacing the rare edge-case scenarios that testers, under deadline pressure, are statistically most likely to miss.

3. ML-Based Defect Prediction

ML-based defect prediction is a machine learning approach that analyses historical code and project data to predict which modules or files in an application are most likely to contain defects, so that limited testing effort can be focused where it is statistically most needed.

Example: Rather than testing all 50 modules of an application with roughly equal effort, an ML model flags the 5 modules most statistically likely to contain a defect based on past history — allowing the team to test those modules first and in the greatest depth, within the same overall time budget.

3.1 Factors ML Analyses

- **Code Complexity:** Measured using metrics such as cyclomatic complexity (essentially, how many independent decision paths a piece of code has). A function containing 15 nested if-statements is harder for a developer to get right, and correspondingly harder to test exhaustively, than a simple linear function.
- **Previous Bugs:** Modules with a history of frequent bug fixes tend to keep generating new defects. A module that needed 12 separate bug fixes over the last year is statistically far more likely to contain the next defect than one that has never been touched.
- **Module History:** Also referred to as "code churn" — how frequently a file or module has been modified, and by how many different developers. A file edited by 8 different developers within a single month signals architectural instability and a higher chance of integration defects.

3.2 The Defect Prediction Pipeline

Turning raw historical project data into an actionable risk score for each module follows a five-stage pipeline:

Step 1 — Collect Historical Data: Source-control commit history, defect-tracking records and code metrics are gathered across the codebase, typically over the last several months or years.

Step 2 — Extract Features: Numerical features are computed for each module — e.g. cyclomatic complexity, historical bug count, and change frequency (churn).

Step 3 — Train the ML Model: A supervised learning algorithm (commonly a Random Forest or Logistic Regression model) is trained on the historical features, learning the relationship between these metrics and where defects actually occurred in the past.

Step 4 — Predict Defect-Prone Modules: The trained model scores every current module, producing a risk classification (e.g. High / Medium / Low) for each one.

Step 5 — Prioritize and Allocate Test Effort: Testers use the risk scores to decide where to invest deep exploratory and regression testing, and where lighter smoke testing is sufficient.

3.3 Worked Example — Module Risk Scoring

Consider an ML model trained on 12 months of commit and defect history for an e-commerce application. It might produce the following risk assessment:

Module	Code Complexity	Past Bugs	Change Frequency	Predicted Risk
--------	-----------------	-----------	------------------	----------------

Payment Gateway	High	12	Very High	High Risk
Login / Authentication	Medium	5	Medium	Medium Risk
Product Search	Medium	3	Medium	Medium Risk
About Us Page	Low	0	Low	Low Risk

Based on this table, testers would concentrate deep exploratory and regression testing on the Payment Gateway module, run moderate coverage on Login and Product Search, and limit the About Us page to a light smoke test — a data-driven allocation of effort rather than an even, and often wasteful, spread across all modules.

Impact of Defect Prediction

Without Prediction	With ML Prediction
All modules tested with roughly equal effort	Test effort is concentrated on the highest-risk modules
High-risk modules may end up under-tested	Defects are caught earlier in the development cycle
Bugs often surface late, close to release	Release risk is quantified with data, not guessed
Test planning relies on tester intuition alone	Test planning is backed by historical evidence

Summary — Topic 3

ML-based defect prediction combines three inputs — code complexity, previous bugs and module change history — to train a model that scores each module's likelihood of containing a defect. The outcome is a smarter allocation of limited testing time: more depth where the data says it matters, less where it doesn't.

4. AI in Requirement Analysis (NLP)

Natural Language Processing (NLP) refers to a family of AI techniques that allow software to read and interpret plain-English text. Applied to requirement analysis, NLP reads requirement documents, user stories and acceptance criteria, extracts structured meaning from them, and uses that understanding both to generate test cases and to flag quality issues in the requirements themselves — before a single line of code is written.

Example: A 40-page Software Requirements Specification (SRS) document that would take a tester a full day to review manually can be scanned by an NLP engine in a matter of minutes to extract every testable rule it contains.

Process Overview

Conceptually, this too follows an inputs → engine → outputs pattern, but the engine here is specifically an NLP pipeline rather than a general ML model:

Step 1 — Input requirement text is collected: Requirement documents, user stories, and acceptance criteria are gathered as plain-text input.

Step 2 — NLP parses and extracts meaning: The engine identifies actors ("who"), actions ("what"), and conditions ("under what circumstance") within each requirement sentence, and separately checks the text for vague or unmeasurable language.

Step 3 — Structured outputs are produced: Draft test cases, ambiguity flags, and automatically compiled test reports are generated for the QA and business-analysis teams to review.

4.1 Core Capabilities

i. Converts Requirement Documents into Test Cases

NLP identifies the actor, action and condition embedded in a requirement sentence, and uses this structure to draft a corresponding test case automatically.

Example: Requirement: "The system shall lock the account after 3 failed login attempts." From this single sentence, NLP auto-generates a test case that verifies the account is indeed locked precisely on the 3rd failed attempt — not the 2nd, and not the 4th.

ii. Identifies Missing or Ambiguous Requirements

Many requirements are written with vague, unmeasurable language that different readers could interpret differently. NLP can detect such wording and flag it for clarification before it causes a defect downstream.

Example: Requirement: "The system should respond quickly." This is flagged as ambiguous, because "quickly" has no measurable threshold. NLP prompts the team to define an exact figure — for example, "the system shall respond within 2 seconds" — before development begins.

iii. Auto-Generates Test Reports

After test execution, AI can automatically compile results — pass/fail counts, coverage percentages, and flagged risk areas — into a shareable report, removing the manual spreadsheet work traditionally required to summarize a test cycle for stakeholders.

4.2 Worked Example — Requirement to Test Case

Requirement: "The system shall allow a registered user to reset their password using a valid email address, and the reset link should expire quickly."

NLP Step	What It Finds	Result
Entity Extraction	Actor = "registered user"; Action = "reset password"; Condition = "valid email"	A clear, testable rule is identified
Ambiguity Detection	The phrase "expire quickly" has no defined time value	Flagged for clarification: exact expiry duration needed
Test Case Generation (Positive)	A registered user resets their password using a valid, registered email	Test case auto-drafted with steps and expected result
Test Case Generation (Negative)	A user attempts to reset a password using an unregistered email	Test case auto-drafted with steps and expected result

The ambiguity flag on "expire quickly" is sent back to the Business Analyst before development even starts. This is significant because catching an unclear requirement at this stage is far cheaper than discovering the same ambiguity as a defect after the feature has already been built and tested.

Summary — Topic 4

NLP-based requirement analysis effectively shifts testing left — ambiguities are caught at the requirement stage rather than after code is written. Its three core outcomes are faster test-case authoring, clearer and more precise requirements, and reporting that requires no manual formatting.

5. Unit Summary & Quick Revision Notes

The four topics in this unit together describe how AI supports software testing across the full lifecycle — from analysing requirements, through generating and executing tests, to predicting where defects are most likely to occur.

Topic	Core Idea
1. Why AI in Testing	AI reduces manual effort in high-volume, change-heavy, maintenance-heavy testing tasks while improving speed, accuracy and coverage.
2. Test Case Generation	AI reads requirements, user stories, defects and UX flow to auto-produce test cases, test data and edge cases.
3. Defect Prediction	ML analyses code complexity, bug history and module change history to predict and prioritize defect-prone modules.
4. Requirement Analysis (NLP)	NLP converts requirement text into test cases, flags ambiguous or missing requirements, and auto-generates test reports.

One-line memory aid for each topic

- AI in Testing → "Test smarter and adapt faster, not just automate more."
- Test Case Generation → "Requirements and history in, test cases and edge cases out."
- Defect Prediction → "Complexity + past bugs + churn = where to look next."
- Requirement Analysis (NLP) → "Fix unclear requirements before they become bugs."